Nullivance Logic: Foundations, Quantified Reasoning, and Certified Recognition

Trinh Tung Lam
lamtung0481@gmail.com

9 November 2026
Part I: Research Overview and Certified Recognition — DR0149-r3

Abstract

When distinct latent states produce the same observations, correct reasoning about those observations need not identify the latent state. We study this separation in Nullivance, a two-channel logic with an optional intensity–structure model. The logical foundation admits exact threshold projection and a complete signed tableau; quantified results cover a finite-domain calculus and a specified pure-equality fragment on finite and infinite carriers. For recognition, we prove that the initial observation cannot determine the defined quasivance predicate. An explicitly stronger scalar probe interface permits exact classification of finite rational histories into infeasible, affirmed, refuted, or undetermined cases, with witnesses and source-backed certificates. We then distinguish compatibility of an infinite history from certification after finite observation. At a positive intensity threshold equal to the latent maximum intensity, finite resolution occurs exactly when an observed lower endpoint contacts the threshold. An unknown period does not suffice for a universally conclusive finite negative procedure; a supplied certified period bound B yields a scanner with at most B contact tests and exact selected-operation counts. Lean 4 checks the stated formal results and implementation refinements. The contribution is a scoped, traceable development connecting semantics, observation limits, and certified algorithms, not a claim to have invented four-valued reasoning or solved general cognition. No empirical performance or cognitive experiment is reported.

Keywords: non-classical logic; formal verification; equality; observability; recognition; certificates; Lean 4.

Reading this collection. This overview connects the project and develops its recognition results. Part II gives the propositional calculus; Part III gives the finite-domain quantified calculus. All three documents carry collection revision DR0149-r3. Archived earlier editions remain available; they are not interchangeable with this edition. The source bundle contains the Lean development, reproducible builds, and a version history.

Contents

1 Introduction **3**
- 1.1 The problem: reasoning from reports is not recovering a state . . . 3
- 1.2 Research questions and results . . . 3
- 1.3 Relation to established work . . . 3
- 1.4 Method and organization . . . 4

2 Two-channel semantics and the propositional core **5**

3 Quantifiers: finite results, infinite obstructions, and equality **6**
- 3.1 Finite-domain proof theory . . . 6
- 3.2 Why finite projection is insufficient for all infinite domains . . . 6
- 3.3 Exact equality procedures . . . 6
- 3.4 Memoization and what the cost theorems measure . . . 7

4 The generative layer: intensity is not observation **7**

5 Recognition as recoverability from observations **8**

6 Scalar probes, noise, and exact finite histories **9**

7 Infinite histories and small certificates **10**

8 Imported memory, trusted continuation, and serialization **11**

9 Threshold observations and finite information limits **12**

10 Adaptive resolution and the exact boundary **13**
- 10.1 A necessary and sufficient finite boundary criterion . . . 13

11 Finite generators, equivalence, and counted cycle bounds **14**

12 Periodicity without a bound is still insufficient **14**

13 Decision with a certified period bound **15**

14 Formal verification and reproducibility **16**

15 Discussion, limitations, and conclusion **17**
- 15.1 What the results mean . . . 17
- 15.2 Outstanding obligations . . . 17
- 15.3 Conclusion . . . 17

A Collection identity and relation to earlier editions **18**

B Complete chronological coverage through DR-0149 **18**

1 Introduction

1.1 The problem: reasoning from reports is not recovering a state

Suppose two instruments disagree about whether a component is active. One may record support for activity without discarding support against it. A logic can specify exactly which conclusions follow from those reports. That task is different from deciding whether the component is actually active: the latter also depends on how the instruments are related to the component. A correct proof system cannot recover information that its observation interface omits. This distinction motivates the present work. The instrument example is an illustration, not an experiment or a validated physical model.

The original Nullivance intuition asks how an unmanifested structure should be distinguished from an absence of structure. We turn this question into a mathematical one: given a latent state space, a target property, and an explicit observation map, which differences between states survive observation? A zero output need not identify its cause. Conversely, declaring a latent coordinate in a model does not demonstrate the existence of a corresponding physical entity. The term *quasivance* below names one precise predicate, not a scientific claim about entities acting without existing.

1.2 Research questions and results

The development addresses three connected questions.

1. *Logical consequence.* Can independent positive and negative support be given exact semantics and a complete proof system? The propositional answer uses threshold projection onto four values and signed tableaux. The quantified answer is restricted: a complete finite-domain calculus and exact algorithms for pure equality, not unrestricted infinite-domain completeness.

2. *Information and certificates.* Which latent properties can be recovered from specified observations? The initial generative observation loses the quasivance distinction. Under stronger scalar probes, compatible states admit an exact box representation. Finite histories then support complete classification and explicit witnesses, including a distinction between inconsistent evidence and genuine uncertainty.

3. *Finite stopping.* When does sufficient information become available after finitely many observations? Infinite compatibility, identifiability from an entire signature, and finite termination are different. Our boundary-contact criterion supports a matched negative/positive pair: unknown periodicity is insufficient for universal finite negative decisions, while a certified upper bound on the contact period gives a correct bounded scanner.

The implementation layer connects these specifications to memoized equality evaluation, checked memory import, resumable replay, and operation counters. The central contribution claimed here is this formally checked chain for the specified models. Standard ingredients and elementary general principles are identified as such; the number of development records is not a measure of novelty.

1.3 Relation to established work

The truth/falsity separation belongs to the Belnap–Dunn and bilattice tradition [1, 5, 6]. The connective called harmonization is the coordinatewise minimum, i.e. bilattice consensus, not a newly discovered algebraic operation. Part II gives the exact fragment embeddings and

literature comparison. Our contribution here is not a claim that classical mathematics cannot describe uncertain or contradictory reports.

Epistemic accounts of distributed systems define knowledge through alternatives compatible with local information [7]. Our observation fiber criterion is a single-observer factorization principle in that tradition. It is not a new modal logic of multi-agent common knowledge. The present target is the specific alpha/Theta predicate and its executable finite evidence contracts.

Representing states consistent with bounded-error data is a standard set-membership estimation problem. Jaulin and Walter's interval approach handles nonlinear set inversion by guaranteed enclosures [8]. Here the enriched probe constraints are rectangular and coordinate-wise, so intersection is exact; we do not claim a solution of general nonlinear identification or superiority to interval-estimation methods.

The sufficient syntactic analyses of regularity are related in method to sound abstraction [3]; failure to certify a property is not its refutation. Ordered reduced Boolean diagrams are due to established work [2]. Our development proves contracts for its particular representations and combinations. This comparison positions the results but is not an exhaustive priority survey of all recognition or verification research.

1.4 Method and organization

We state a model and its input contract, prove a result or construct a counterexample within that model, and link the statement to a Lean declaration. Implementation refinements have separate correctness and cost claims. The method is mathematical proof and formal verification, not experimental confirmation of the model. Worked numerical cases illustrate definitions; they do not replace universally quantified proofs.

Three different objects must be kept separate. The first is a logic of independent support for and against propositions. The second is a generative model for the values supplied to that logic. The third is an observation model in which an observer attempts to recover specified properties of a latent state. A theorem about one object is not automatically a theorem about the others. In particular, evidence in both channels is permitted by the logic; factive knowledge of a proposition and its negation is not.

The exposition proceeds from logic and quantification to the latent model, finite observations, infinite histories, and stopping. Part II and Part III supply the full tableau rule sets and their detailed proof arguments, so the three-part collection should be used for those dependencies. This overview is not a replacement for those calculi. Appendix B is a provenance index, separate from the mathematical argument. Full Lean types retain all semantic and typeclass hypotheses; resolving a theorem name alone does not verify a natural-language paraphrase.

Layer	Established scope; important exclusion
Propositional core	Signed tableau soundness/completeness and exact threshold projection; no interpretation as a physical law.
Quantification	Finite-domain core tableau; finite/infinite pure-equality algorithms; not arbitrary-predicate infinite-domain completeness.
Recognition	Exact properties of specified observations of a fixed scalar state; not unrestricted perception, changing-state cognition, or learning.
Algorithms	Correctness and explicitly charged operations of named programs; not a universal wall-clock, memory, or bit-complexity bound.
Publication evidence	Compiling proof artifact, dependency audits, source manifest, and reproducibility instructions; not external peer review.

Definitions stipulate a model; proofs establish consequences of that model. Any application must independently justify its observation map, fixed-state assumption, error bounds, and

supplied stopping certificates. These are not provided by the logical connectives or by the proof assistant.

2 Two-channel semantics and the propositional core

An atomic proposition receives a pair $(t, f) \in [0,1]^2$. These coordinates are independent support values, not complementary probabilities. For values $x = (t, f)$ and $y = (u, v)$, the primitive operations are

$$\neg x = (f, t), \quad x \wedge y = (\min(t, u), \max(f, v)), \quad x \vee y = (\max(t, u), \min(f, v)),$$

and harmonization (consensus) is $x \oplus y = (\min(t, u), \min(f, v))$. A threshold $0 < \tau \leq 1$ records whether each coordinate is at least τ. The four resulting Boolean pairs are $\mathrm{N} = (0, 0)$, $\mathrm{T} = (1, 0)$, $\mathrm{F} = (0, 1)$, and $\mathrm{B} = (1, 1)$. Here zero and one in a Boolean pair denote bits. The signs T^+, T^-, F^+, F^- assert, respectively, positive truth support, its absence, positive falsity support, and its absence at the specified threshold. Absence of support is not identified with support for the opposite proposition.

Example 2.1. *At $\tau = 1/2$, $(0.8, 0.7)$ projects to B, whereas $(0.2, 0.1)$ projects to N. The first can represent conflicting reports; the second insufficient reports. Neither description says that an observer knows both a fact and its negation. The same proposition can have different reports without changing the mathematical meanings of the two channels.*

Theorem 2.2 (Exact propositional projection)**.** *Thresholding the continuous evaluation of a formula gives exactly its four-valued evaluation under thresholded atomic values.*

Proof. Induct on the formula. Swapping coordinates commutes with thresholding. For two reals, a minimum is at least τ iff both operands are; a maximum is at least τ iff at least one is. These identities establish all three binary cases coordinate by coordinate. This argument uses finite min/max, not an unattained supremum of an infinite family. □

Lean anchor: `Nullivance.Continuous.exact_projection`.

The analytic signed tableau closes precisely the unsatisfiable finite signed branches. The corresponding consequence theorem transfers to the continuous semantics by exact projection. The artifact also supplies executable reference checking, countermodel extraction on failure, and compactness/strong-completeness results for infinite premise sets. The last phrase concerns *sets of propositional premises*; it does not mean arbitrary quantified object domains.

Lean anchor: `Nullivance.Metatheory.Closes.unsat`.

Lean anchor: `Nullivance.Metatheory.closes_of_unsat`.

Lean anchor: `Nullivance.Metatheory.derives_iff_consequenceC`.

Paraconsistency means that contradictory support need not entail an arbitrary unrelated proposition. It does not make every inference permissible. The two companion papers contain the detailed core calculus and finite-domain tableau exposition. Part II distinguishes the paper-level coNP classification from its machine-checked certificate and reduction components; its FDE and bilattice connections have explicit formal anchors. These statuses are not interchangeable. The mathematical core deliberately builds on established four-valued and bilattice ideas [1, 5, 6]. The availability of a Lean formalization alone does not establish historical novelty of those ideas.

3 Quantifiers: finite results, infinite obstructions, and equality

3.1 Finite-domain proof theory

The finite-domain work treats a function-free quantified language with crisp equality and a nonempty finite domain. The macro-free extensional core tableau has a soundness and completeness theorem. Equality substitution and replay require their actual admissibility conditions; earlier proposed replay routes were corrected rather than left as assumed lemmas. Part III distinguishes those failed routes from the proved final calculus. A refuted auxiliary conjecture does not refute the subsequently repaired theorem.

Lean anchor: `Nullivance.FiniteFO.qDerivesExtCore_iff_qconsequence4Sig`.

3.2 Why finite projection is insufficient for all infinite domains

Consider truth values $a_n < \tau$ increasing to τ. The existential truth coordinate may be $\sup_n a_n = \tau$, so its continuous threshold bit is true, although every individual threshold bit is false. Moving projection through this supremum is invalid. Positive results therefore require stated witness attainment, threshold regularity, or suitable compactness and semicontinuity hypotheses. Equality depending on a bound variable also matters: an arbitrary crisp equality predicate need not have the topological regularity that an informal continuity argument assigns to it.

The abstract analyses track constant, crisp, and upper-semicontinuous coordinates, then coordinate relations and exact complementarity. These are sound sufficient recognizers, not complete recognizers for every semantic property. Boolean normalization by ROBDD removes a particular syntactic limitation: equivalent Boolean combinations need not have the same syntax tree. But a propositional ROBDD treats equality atoms as independent unless the equality theory is added. For example, reflexivity of $x = x$ and transitive consequences of $x = y, y = z$ are theory facts, not arbitrary Boolean tautologies. Reduced ordered decision diagrams are an established Boolean representation [2]; the present artifact verifies their use in its own pipeline, not their invention.

3.3 Exact equality procedures

The equality branch adds equivalence classes, disequality constraints, binder-aware assignment updates, and fresh-element capacity. Universal and existential choices are represented by existing classes and, when possible, a fresh class. Forgetting which variables are currently bound can merge semantically different calls; the cache keys therefore retain the necessary scope/capacity information. A cutoff argument replaces infinitely many indistinguishable choices by finitely many equality types.

The resulting total decision procedures are exact for the stated *pure-equality fragment*. For closed formulas the artifact proves correctness on infinite carriers and on nonempty finite carriers with the appropriate capacity. It does not decide arbitrary predicates on an infinite domain. The later open-formula interfaces evaluate a *supplied assignment*, not automatically every assignment.

Lean anchor: `Nullivance.InfiniteFO.decideQuantifiedEqualityHashed_infinite_correct`.

Lean anchor: `Nullivance.InfiniteFO.decideQuantifiedEqualityHashed_finite_correct`.

For an open formula of quantifier rank r, let c be the number of distinct values assigned to its relevant free variables. Compact encodings use their equality classes and an explicit fresh pool. The sufficient finite capacity condition is $r + c \leq |D|$; when it fails, the checked finite interface can use a supplied complete duplicate-free enumeration of the actual domain. The validation of the enumeration and fragment is part of the interface. Rejection of an unsupported

input is not a false truth value. The conservative compact representative construction uses $r + c + 1$ elements; no minimality of that count is asserted here. Enumerating validity across all free assignments remains separate from evaluating one assignment.

Lean anchor: `Nullivance.InfiniteFO.decideFiniteOpenEquality_correct`.

Lean anchor: `Nullivance.InfiniteFO.checkedFiniteOpenEquality_sound`.

Lean anchor: `Nullivance.InfiniteFO.decideFreshCompactOpenEquality_finite_capacity_correct`.

3.4 Memoization and what the cost theorems measure

The optimized evaluator is proved equivalent to its reference recursion; the table is threaded through recursive orbit expansion. Its invariant identifies distinct stored states with misses from the specified empty-table execution. Syntax-sensitive Bell-number and factorial bounds then control the number of unique keys. The exact bound parameters and closure conditions are retained in the following source anchors, not suppressed into an unqualified "polynomial-time" claim. Writing r for quantifier rank and $|\varphi|$ for syntax-node count, the empty-root specialization proves

$$U = \mathsf{misses} \leq |\varphi|\, B_r \leq |\varphi|\, r!,$$

where B_r is the artifact's equality-partition (Bell) count at that root. The stronger depth-sensitive envelope is available in the source; the simple display is an upper bound, not the exact count of every run.

Lean anchor: `Nullivance.InfiniteFO.runQuantifiedEqualityOrbitHashed_correct`.

Lean anchor: `Nullivance.InfiniteFO.runQuantifiedEqualityOrbitHashed_unique_states_eq_misses`.

Lean anchor: `Nullivance.InfiniteFO.runQuantifiedEqualityOrbitHashed_unique_states_le_syntax_mul_bell`.

Lean anchor: `Nullivance.InfiniteFO.runQuantifiedEqualityOrbitHashed_unique_states_le_syntax_mul_factorial`.

For the pinned Lean implementation, a bucket-doubling potential bounds rehash work linearly in the number U of inserted unique states. In the instrumented model, rehashed entries are bounded by $3U$ and the counted hash operations by $A + 4U$, where A is the counted request quantity in the formal cost specification. Collision-chain comparison costs retain their dependence on chain length; no random hash distribution is postulated. The displayed amortized operation bounds hold for the instrumented root run; the syntax/Bell envelope specializes the parameter to quantifier rank. Semantic correctness additionally requires the stated fragment and closure conditions. These costs exclude the bit cost of a large key, rational arithmetic, allocation, and the entire surrounding ROBDD algorithm.

Lean anchor: `Nullivance.InfiniteFO.runQuantifiedEqualityOrbitHashed_amortized_state_cost_bounds`.

4 The generative layer: intensity is not observation

A generative frame has a positive structure dimension d, a stability map $\Phi : [0,1]^d \to [0,1]$, flip symmetry $\Phi(1 - \Theta) = \Phi(\Theta)$, and neutral value $\Phi(1/2, \ldots, 1/2) = 1$. A channel contains an intensity $\alpha \in [0,1]$ and a structure $\Theta \in [0,1]^d$. Its effective output is $\alpha\Phi(\Theta)$. A state has two channels and initializes the core value as

$$\mathsf{init}(s) = (\alpha_+ \Phi(\Theta_+), \alpha_- \Phi(\Theta_-)).$$

One concrete admissible family uses the geometric mean of componentwise $1 - 2|\theta - 1/2|$. This is a model definition, not a measured biological law.

Lean anchor: `Nullivance.Generative.GenState.init_mem`.

Definition 4.1 (The specified latent predicate)**.** *A channel is quasivant when $\alpha = 0$ and its structure is not neutral. A state is quasivant, written $\mathsf{Q}(s)$, when both channels are quasivant.*

This exact conjunction is important. Small positive intensity is not zero; a neutral structure is not nonneutral; one quasivant channel is not two. All quasivant states initialize to $(0, 0)$, but the converse fails.

Proposition 4.2 (Silent ambiguity in every admitted frame)**.** *There are a quasivant state and a nonquasivant state with identical initial observations, for every admitted generative frame.*

Proof. Set both intensities to zero. In one state set both structures constantly to zero, and in the other constantly to $1/2$. Positive dimension ensures these structures differ. The first state satisfies the defined predicate and the second does not. Multiplication by zero makes both observations $(0, 0)$, independently of the admitted stability map. □

Lean anchor: `Nullivance.Recognition.every_frame_silent_ambiguity`.

This refutes universal recovery of this latent predicate from init alone. It neither disproves the existence of structured latent states nor requires changing the logical core.

5 Recognition as recoverability from observations

Let $o : S \to O$ be an observation map and $P : S \to \mathsf{Prop}$ a property. Exact recognizability means that a decoder $d : O \to \mathsf{Prop}$ exists with $d(o(s)) \leftrightarrow P(s)$ for every state.

Theorem 5.1 (Fiber criterion)**.** *Such a decoder exists iff P is constant on each observation fiber:*

$$o(s) = o(t) \implies (P(s) \leftrightarrow P(t)).$$

Proof. A decoder gives the forward implication immediately. Conversely define $d(y)$ by the existence of a state s with $o(s) = y$ and $P(s)$. At any actual observation $o(t)$, fiber constancy makes this equivalent to $P(t)$. This constructs a mathematical predicate, not necessarily a computable test. □

Lean anchor: `Nullivance.Recognition.recognizable_iff_fiber_constant`.

Postprocessing cannot restore distinctions already removed by the observation. Extra observations can preserve or improve recoverability. A family of probes supports recognition iff it separates every pair disagreeing on the target property; observing an entire possibly infinite signature is not the same as finding a finite stopping rule.

Lean anchor: `Nullivance.Recognition.recognizable_of_postprocess`.

Lean anchor: `Nullivance.Recognition.recognizable_tests_iff_separates`.

If deterministic dynamics preserve observational equality at each next step, induction preserves it along the whole trace. Merely appending a time axis does not then recover the lost distinction. This is a conditional result about observation-stable dynamics, not an impossibility for all interventions. A coordinate-swap example shows how an intervention can reveal a hidden bit when that stability condition is absent.

Lean anchor: `Nullivance.Recognition.stable_evolution_indistinguishable`.

Factive knowledge is separately specified by $\mathsf{Known}_o(P, s) \leftrightarrow \forall t,\ o(t) = o(s) \Rightarrow P(t)$. The actual state is a compatible alternative, so knowledge implies truth. Four-valued support is not automatically knowledge. Even T alone is insufficient without a sound connection to the target, and B cannot be promoted factively to both a proposition and its negation. A positive bridge requires adequacy of reports and constancy on the observation fiber.

Lean anchor: `Nullivance.Recognition.truthSupport_known_of_adequate`.

Lean anchor: `Nullivance.Recognition.bothSupport_no_unconditional_knowledge`.

These hypotheses are application obligations, not facts silently supplied by the Nullivance connectives.

6 Scalar probes, noise, and exact finite histories

The constructive recognition branch specializes the generative frame to four scalar coordinates $(\alpha_+, \alpha_-, \theta_+, \theta_-) \in [0, 1]^4$ and an enriched probe interface. Probe readings expose information absent from init alone; this additional observation model is not derived by decoding init. Concretely, the intensity intervention replaces structure by neutral and retains intensity, so its response is exactly α. In the canonical one-dimensional frame, the structure intervention sets intensity to one and halves the original structure coordinate; its response is exactly θ. These experiments operate on separate copies of the same original state. Their combined exact-real signature is injective, hence information-theoretically sufficient for any property of that state. The signature is not an executable decision procedure for every predicate over real numbers, and the proposed interventions have not been implemented as physical experiments.

Lean anchor: `Nullivance.Recognition.activeProbeSignature_eq`.

Lean anchor: `Nullivance.Recognition.activeProbeSignature_injective`.

The scalar target is

$$\mathsf{Q}(s) \ \leftrightarrow\ \alpha_+ = 0 \wedge \alpha_- = 0 \wedge \theta_+ \neq 1/2 \wedge \theta_- \neq 1/2.$$

A rational observation records a rational allowance ϵ and four rational coordinates y_i. Compatibility is the stated coordinatewise error contract, together with the admitted unit bounds and allowance conditions. All observations in one history must fit *the same* latent state.

For a well-formed finite history, compatible coordinates lie in a rectangular box with endpoints

$$L_i = \max\bigl(0, \max_j(y_{ij} - \epsilon_j)\bigr), \qquad U_i = \min\bigl(1, \min_j(y_{ij} + \epsilon_j)\bigr).$$

The empty history starts with $[0, 1]^4$. Malformed allowances and empty intersections are handled by the implemented feasibility checks; the formula above is not permission to ignore input validity.

Lean anchor: `Nullivance.Recognition.summarizeProbes_exact`.

Lean anchor: `Nullivance.Recognition.historyFeasible_iff_exists`.

Theorem 6.1 (Exact finite classification)**.** *Every finite input history receives precisely one of: invalid (no compatible state), affirmed (nonempty and all compatible states quasivant), refuted (nonempty and all compatible states nonquasivant), or undetermined (compatible states of both kinds).*

Proof. For each coordinate retain the two interval endpoints and its distinguished marker: zero for intensity, $1/2$ for structure. This gives $3^4 = 81$ candidate entries, not necessarily distinct or feasible. Search only candidates passing all bounds and the requested truth value of

Q. Every real compatible state has a candidate preserving the four equalities that determine Q; conversely each accepted candidate embeds into an admitted state. The two existence flags determine the four disjoint cases. Feasibility prevents vacuous affirmation on inconsistent data. □

Lean anchor: `Nullivance.Recognition.boxWitnessCandidates_represent`.

Lean anchor: `Nullivance.Recognition.exportHistoryWitness_realizes`.

Lean anchor: `Nullivance.Recognition.classifyHistory_complete`.

Example 6.2 (Reading the four verdicts)**.** *Consider boxes with both structure coordinates in* $[0, 1/4]$*, so neither can be neutral. If both intensity intervals are* $[0, 0]$*, the box is nonempty and every member satisfies* Q*: the verdict is affirmed. If the positive intensity interval is* $[1/10, 1/5]$*, every member violates* Q*: refuted. If both intensities instead range over* $[0, 1/10]$*, zero intensities and positive intensities are both possible: undetermined. Finally, contradictory constraints requiring one coordinate to be at most* $1/4$ *and at least* $3/4$ *have no common state: invalid. These elementary box examples illustrate the classifier's specification, not empirical measurements.*

An undetermined verdict therefore contains positive mathematical information: both kinds of state remain possible. An invalid verdict identifies failure of the common-state/error contract and must not be interpreted as evidence for every property. These four verdicts describe sets of compatible latent states; they are not a renaming of the logical values N, T, F, B.

The rational witness is actual exported data, not a claim that an arbitrary real hidden state can be computed. Completeness here concerns this predicate and independent coordinate boxes. Correlated nonlinear constraints would require another representation theorem. Permuting or duplicating observations does not alter the compatible set; the artifact proves the corresponding implementation invariances.

Example 6.3 (Combining ambiguous records)**.** *With allowance* $1/10$*, readouts* $((0, 0), (2/5, 0))$ *and* $((0, 0), (3/5, 0))$ *are separately undetermined. Together their positive-structure interval is exactly* $[1/2, 1/2]$*. Every common state has a neutral positive structure, so the combined verdict is refuted, not invalid.*

Lean anchor: `Nullivance.Recognition.joint_neutrality_refutation_regression`.

The example illustrates intersection of evidence about one state; it does not model an unannounced change of state between readings.

7 Infinite histories and small certificates

For the same scalar, closed, independent-coordinate model, a single state fits a countable stream iff every finite prefix is feasible. In fact it is enough that every pair of records has a common fitting state.

Argument for the compatibility result. Pairwise feasibility implies each lower endpoint is at most every upper endpoint in the same coordinate. Take the supremum of all lower endpoints. The set is nonempty and bounded above by an upper endpoint from record zero. Its supremum lies between all the lower and upper endpoints and in $[0, 1]$. The four coordinates assemble one admitted state fitting the entire stream. The reverse directions follow by restricting a fitting state. □

Lean anchor: `Nullivance.Recognition.stream_exists_iff_all_prefixes_feasible`.

Lean anchor: `Nullivance.Recognition.stream_exists_iff_pairwise_feasible`.

Thus inconsistency has a certificate of at most two observations. Two is sharp: exact positive-intensity readings zero and one are individually feasible but jointly inconsistent. This is a rectangular-constraint result, not a new unrestricted compactness theorem. Certificate size does not bound where the records occur or how long the observer waits.

Lean anchor: `Nullivance.Recognition.stream_infeasible_iff_two_observation_certificate`.

Lean anchor: `Nullivance.Recognition.two_observation_bound_is_sharp`.

Compatibility compactness must not be confused with finite certification of every universal property. An infinite shrinking history can force a boundary value while every finite prefix still admits alternatives. The development therefore preserves explicit non-stopping examples, instead of assuming that infinite evidence always becomes finite evidence.

The certificate branch makes these distinctions operational. Endpoint summaries carry the original observations responsible for extremal bounds. Conflict, affirmative, and refutation certificates are checked against source positions. Feasibility is retained for non-vacuous positive or negative conclusions. An actual source index is more informative than an unattributed value: a numerically correct endpoint alone does not authenticate its claimed origin. The certificate and memory refinements develop extraction, indexed provenance, joint pipelines, cursor-field costs, and short-circuit equality refinement. Each cost theorem refers to its own instrumented implementation; output equivalence alone is not used to transfer an unproved runtime bound.

8 Imported memory, trusted continuation, and serialization

A saved cursor includes its next position and a sourced summary. On import, the externally supplied object is not assumed to match the underlying records. The checked entry recomputes the prefix summary and compares it with the imported value. It returns a proof-carrying handle precisely when `CursorMatches` holds. This invariant concerns equality to the summary of the designated source prefix; it is not cryptographic proof that an external provider reported physical truth.

Lean anchor: `Nullivance.Recognition.importTrusted_isSome`.

Lean anchor: `Nullivance.Recognition.importTrusted_output`.

Proposition 8.1 (Counted import)**.** *For cursor position n, the specified counted importer equals the checked import result together with $8n$ rational-order nodes and one whole-box equality call, whether it accepts or rejects.*

Proof. The counted source-prefix check has the same Boolean result as the ordinary check and the displayed counters. The importer branches on that Boolean: acceptance packages the same cursor with the established matching proof; rejection returns no handle. Neither branch changes the counters. Substitution of the prefix-check specification therefore gives the result in both cases. The proof field is a logical guarantee, not additional serialized evidence. □

Lean anchor: `Nullivance.Recognition.importTrustedCounted_spec`.

The unit "one equality call" is deliberately not one machine instruction: later refinements count fields and short-circuit behavior separately. Once accepted, an in-process proof-carrying continuation preserves the invariant without rescanning the old prefix at every step. The stream is fixed during these proofs. Mutating the source after validation is outside the contract.

A finite digest cannot universally replace exact checking: any map from the relevant infinite family into a finite digest space collides. The formal counterexample includes feasible histories, so the obstruction is not merely malformed data. This is an impossibility of *unconditional exact inference* from a finite digest, not a refutation of probabilistic cryptographic security under a specified threat model.

Lean anchor: `Nullivance.Recognition.finite_digest_cannot_justify_import`.

The serialization layer specifies session lifecycle and checked byte/envelope boundaries. A Lean structure carrying a proof is not by itself a sealed runtime capability: proof fields are erased. Reimporting raw data requires checking again or an explicitly specified trusted-storage assumption. The later cache, checkpoint, and typed-phase imports likewise validate the relevant semantic invariants rather than treating serialization success as semantic correctness. The project does not yet prove a general secure distributed-storage system.

Lean anchor: `Nullivance.Recognition.decodeCursorEnvelope_encode`.

Lean anchor: `Nullivance.Recognition.restoreActiveEnvelopeCounted_success`.

The separate threshold-endpoint cache validator also has an exact acceptance specification relative to rebuilding from supplied threshold data. Its accepted path costs $2n + 11$ selected operations when $n < 2$, and $2n + 18$ otherwise. The model charges list-constructor inspections, stage-tag inspections, and Boolean gates; its comparison circuit is charged without short-circuit savings. This is a different cost unit from the rational-order nodes of sourced-history import. Acceptance does not imply that arbitrary supplied threshold data are realizable by an underlying state, nor that two full histories are equal.

Lean anchor: `Nullivance.Recognition.checkThresholdCache_spec`.

Lean anchor: `Nullivance.Recognition.checkThresholdCacheCounted_accepted`.

Lean anchor: `Nullivance.Recognition.checked_cache_nonrealizable_counterexample`.

9 Threshold observations and finite information limits

There are two distinct information models: exact or bounded-error scalar readings, and observations restricted to threshold bits. Finitely many positive thresholds cannot in general distinguish exact zero from every sufficiently small positive intensity. An appropriate infinite countable threshold family can determine zero as a mathematical property of the entire signature, but this does not produce a finite algorithm affirming zero in every case. The quasivance predicate additionally requires both structures to be nonneutral. An intensity-only result cannot settle that conjunction.

Lean anchor: `Nullivance.Recognition.finite_positive_thresholds_not_recognizable`.

Lean anchor: `Nullivance.Recognition.countableThreshold_recognizes_quasivance`.

The threshold analysis distinguishes init-only and partial-probe losses, finite versus countable threshold signatures, margin assumptions, neutrality boundaries, finite refutation, and endpoint compression. The finite-prefix classifier retains its distinction between invalid, affirmed, refuted, and undetermined. A below-threshold status is not equality to zero and is not automatically affirmation of quasivance. Conversely an above-positive-threshold certificate refutes quasivance by witnessing positive intensity in at least one channel.

The streaming implementation connects these information limits to supplied-list costs, incremental summaries, checked imported caches, first-hit stopping, consumed-prefix accounting, chunk composition, checkpoint continuation, session ledgers, ordered delivery, and explicit precision deadlines. "Eventually delivered" is a hypothesis about records received by the algorithm, not a theorem that a physical sensor will supply them. Selective refresh preserves

resolved results only under the proved refinement and feasibility conditions. Successful chunk composition does not authorize reordering records or skipping the source-prefix checks.

10 Adaptive resolution and the exact boundary

Let $I(s) = \max(\alpha_+, \alpha_-)$. A finite list of positive rational thresholds is separated from this value when $I(s) \neq \tau$ for each listed τ. Such a finite separated list admits a common positive rational margin. Existence of that margin is a proof fact, not a decoder extracting it from arbitrary observations. With the specified delivery of arbitrarily fine precision, the bounded reference search eventually finds a prefix resolving all requested thresholds. It does not receive the latent state, margin, or delivery schedule as executable inputs.

Lean anchor: `Nullivance.Recognition.separated_stream_adaptive_resolution`.

The search starting at prefix length a with fuel b tests lengths $a, \ldots, a + b - 1$. It returns the first ready prefix or `none`. Here `none` means no success within that window, not a negative theorem about the latent state. Zero fuel tests nothing. Empty threshold lists still require feasibility. The executable refinements implement this reference search as incremental, whole-run and read-on-demand replay, including shortage, resume, typed phase import, and compact initial-prefix summaries. These refinements preserve the reference first-success result and separately account for the operations in their instrumented implementations. The compact initialization summary retains eight rational endpoints and at most eight occupied observation/index slots. This bound is attained for that representation; it is not a lower bound for every algorithm. Equal sourced summaries give equal future phases for fixed requests, fuel, and a common ordered suffix. They do not determine the original history: different records can yield the same summary. Thus a correct resumable memory is not a lossless archive of research data.

Lean anchor: `Nullivance.Recognition.compactReplay_extension`.

Lean anchor: `Nullivance.Recognition.bootSources_bound`.

Lean anchor: `Nullivance.Recognition.bootSources_eight_attained`.

Lean anchor: `Nullivance.Recognition.no_bootSummary_history_inverse`.

10.1 A necessary and sufficient finite boundary criterion

For one observation $r_j = (\epsilon_j, y_j)$ define its intensity floor

$$f(r_j) = \max(y_{+,j} - \epsilon_j,\ y_{-,j} - \epsilon_j).$$

Every fitting state's maximum intensity is at least this floor. Suppose a common fitting state has $I(s) = \tau > 0$. Then a finite history resolves this threshold iff some observed floor equals τ.

Lean anchor: `Nullivance.Recognition.boundary_ready_iff_contact`.

Proof. The feasible box certifies "at least τ" precisely when one lower intensity bound reaches τ. Its initial value zero cannot do so because $\tau > 0$. Every later lower bound is the maximum of prior lower bounds and observed lower endpoints. Each such endpoint is at most the fitting intensity, hence the largest floor is at most $I(s) = \tau$. Reaching the threshold therefore means equality at an observed floor. A below verdict is excluded by the given fitting boundary state. If there is no contact, the simultaneous lower corner of the feasible box has both intensities below τ and is itself a fitting state. The finite history then admits both a boundary state and a below-threshold alternative. □

Lean anchor: `Nullivance.Recognition.boundary_contact_or_alternative`.

For a threshold in $(0, 1]$, constant centered intensity readings with allowances $1/(j+1)$ can force $I = \tau$ in the whole infinite stream while every finite search times out: each floor remains strictly below τ. Arbitrarily fine precision alone is therefore insufficient for universal finite boundary resolution. Conversely the reading $y_+ = 3/4$ with allowance $1/4$ has floor $1/2$ and certifies at least $1/2$. Exact zero noise is not necessary.

Lean anchor: `Nullivance.Recognition.centeredBoundary_timeout`.

Lean anchor: `Nullivance.Recognition.noisy_contact_regression`.

The successful noisy reading certifies an inequality, not exact equality to $1/2$. Equality is an assumption of the boundary analysis, not a conclusion silently inferred from that reading.

11 Finite generators, equivalence, and counted cycle bounds

A contact generator consists of a start state, a deterministic next-state function, and a Boolean mark. With a finite state space of size D, if a mark is ever reached then one is reached before step D. A bounded scanner returns the first marked position; exhaustion is a permanent negative only when the applicable cutoff theorem has been established.

Generator descriptions can provide information absent from sampled traces. The development includes finite-product trace comparison, checked finite relations, symbolic simulation relations, and residue-counter encodings. A symbolic relation certificate can establish equality of all output traces, but existence of such a relation is not an algorithm discovering it. Similarly an affine-residue checker is complete for the proposed affine invariant it checks, not for every pair of trace-equivalent generators. Unreachable states can invalidate that invariant without changing reachable output behavior.

For a residue counter with modulus $P > 0$ and stride s, the reachable cycle length is $c = P/\gcd(P, s)$. In particular stride zero gives $c = 1$, not zero. Two such traces with cycle lengths c, d can be compared through $L = \operatorname{lcm}(c, d)$, bounded by the earlier product cutoff. The counted comparison preserves the exact earliest mismatch. The cost analysis explicitly instruments a Euclidean algorithm for setup, proves a logarithmic remainder-step bound by two-step halving, and avoids setup entirely when the first marks already differ.

For the specified setup, if $S = \log_2 P + \log_2 Q + \log_2 c$ with natural-number logarithms, the proved selected-work envelope is $2S + 13 + 2L$. This counts the named remainder/division/product/mark/transition operations. It is not the bit complexity of Euclid or the runtime of any uninstrumented built-in gcd implementation. A large-number division has not been proved to cost one processor step. Equivalence to a faster-looking algorithm does not change this scope.

Lean anchor: `Nullivance.Recognition.compareResidueCycles_exact`.

Lean anchor: `Nullivance.Recognition.euclidCounted_log_bound`.

Lean anchor: `Nullivance.Recognition.preparedCycleWork_log_bound`.

Lean anchor: `Nullivance.Recognition.guardedCycles_result`.

12 Periodicity without a bound is still insufficient

Here periodicity refers to the Boolean contact pattern, not necessarily the full rational observation stream. A residue-generator description enables a cutoff calculation. Merely promising some unknown period to an observation-only algorithm does not supply that calculation.

Theorem 12.1 (Finite negative query obstruction)**.** *There is no deterministic observation-only strategy that finitely returns a sound negative answer exactly on every never-contacting stream in the specified periodic encoding family, without being supplied a period bound. Queries may be adaptive, repeated, or out of order.*

Finite-transcript argument. Run the strategy on the silent encoded counter. Any terminating negative run has queried finitely many indices. Choose N larger than all of them. A late-contact periodic counter with cycle $N+1$ agrees on every queried observation but first contacts at N. Determinism preserves the same transcript and negative answer, contradicting soundness on this later-contact stream. The encoding maintains a common admitted scalar state and the stated precision properties, so the counterexample is inside the modeled class. □

Lean anchor: `Nullivance.Recognition.periodicQuery_admissible_collision`.

Lean anchor: `Nullivance.Recognition.no_finite_negative_periodic_recognizer`.

This is not a theorem about randomized algorithms, arbitrary source-code analysis, or uncomputability of every particular generator. It also does not say positive contacts are undiscoverable: a finite observed contact is a positive certificate. The obstruction concerns universally conclusive finite *negative* observation-only decisions under the stated interface.

13 Decision with a certified period bound

Define $C(n)$ by $f(r_n) = \tau$. The new assumption is

$$\mathsf{ContactPeriodBound}(r, \tau, B): \quad \exists P > 0, \quad P \le B \ \wedge \ \forall n, \big(C(n+P) \leftrightarrow C(n)\big).$$

This is an all-index semantic promise, not a Boolean test performed on the first B samples. The executable scanner receives r, τ, B, not the period or generator description. It checks indices $0, \dots, B-1$ in order and stops at the first contact.

Theorem 13.1 (Certified cutoff and scanner specification)**.** *Under the period-bound promise, contact ever occurs iff it occurs at some $k < B$. The scanner returns $\mathsf{some}(k)$ exactly when $k < B$, $C(k)$, and no earlier index contacts; it returns none exactly when contact never occurs. The first-hit specification needs no periodicity assumption; the permanent-negative specification does.*

Proof. If a contact exists, take its least index m. If $m \ge P$, periodicity yields a contact at $m - P < m$, a contradiction. Hence $m < P \le B$. The reverse implication is immediate. The bounded scanner's first-hit and finite-exhaustion specifications follow from its recursive mark test. Composing finite exhaustion with the cutoff yields the permanent-negative statement. □

Lean anchor: `Nullivance.Recognition.contactPeriodBound_cutoff`.

Lean anchor: `Nullivance.Recognition.scanObservedContact_some`.

Lean anchor: `Nullivance.Recognition.scanObservedContact_permanent`.

If all observations fit one admitted scalar state with $I(s) = \tau > 0$, the negative answer is equivalent to the canonical threshold procedure never finding a ready prefix. A first contact at k gives its first successful prefix $k+1$, tested with budget $B+1$ because the empty prefix is included. No precision-delivery assumption is required for this exact-boundary bridge. The negative result means "no finite boundary certificate in this procedure," not "the intensity is below τ": the theorem assumes equality.

Lean anchor: `Nullivance.Recognition.scanObservedContact_boundary`.

Lean anchor: `Nullivance.Recognition.scanObservedContact_first_prefix`.

Theorem 13.2 (Exact selected-operation counts)**.** *The counted scanner returns the same result as the scanner. At a first hit k, it charges $(k+1, k)$ contact tests and successor transitions. At exhaustion it charges (B, B). Both counters are at most B.*

Proof. Instantiate the counted generator scanner with the actual rational floor equality as its mark operation and natural successor as its step. Each failed test advances once, including the final failing transition on exhaustion; a successful test does not advance. The exact recurrence gives the two cases and, since $k < B$, the bounds. □

Lean anchor: `Nullivance.Recognition.scanObservedContactCounted_exact`.

Lean anchor: `Nullivance.Recognition.scanObservedContactCounted_boundary`.

This charges the operations of the actual wrapper, not a cost inferred merely from output equivalence. It excludes sample production, rational bit arithmetic, allocation, and compiler-level callback behavior. Computability is relative to an evaluable stream; an arbitrary mathematical function is not a hardware sensor driver.

The late-contact family proves sharpness of this leading-prefix bound: with period $N + 1$, N observations return no hit, whereas $N + 1$ observations first hit at N. This is not an adaptive-query lower bound for the bounded class. Budget zero is executable and returns no hit with zero work, but it cannot satisfy the positive-period promise. Testing a few samples for apparent periodicity does not justify promoting exhaustion to a permanent negative.

Lean anchor: `Nullivance.Recognition.scanObservedContact_cutoff_sharp`.

Lean anchor: `Nullivance.Recognition.contactPeriodBound_positive`.

14 Formal verification and reproducibility

The verification uses Lean 4 [4]. The companion artifact pins the mathematical baseline, Lean toolchain, mathlib commit, and all dependency revisions in the Lake manifest. Its `SOURCE_MANIFEST.sha256` covers packaged files; a separate digest covers the source ZIP. Hashes detect byte changes relative to that manifest; they are not proofs of scientific truth or external signing credentials.

The verification script runs the library build, both research validation entry points, and the overview and companion declaration/type/dependency audits. It rejects proof holes and nonstandard axioms in the report's theorem anchors. The complete validation logs remain available for inspection. Legacy executable regressions that use a native decision bridge must not be confused with the structural mathematical proofs: any such dependency is disclosed in the validation inventory rather than erased from the record. The latest recognition boundary results use kernel-checked proofs and standard Lean logical dependencies only.

The expected standard logical dependencies are `propext`, `Classical.choice`, and `Quot.sound`, or subsets. "No added axioms" does not mean that classical mathematics has no foundational assumptions. Axiom audits inspect the dependency closure of a declaration; source-token scans alone would not suffice.

The theorem-anchor audit prints full elaborated types. This permits checking that, for example, a theorem is conditional on `StreamFits` and `ContactPeriodBound`, rather than simply checking that its name exists. The publication review additionally inspects definitions, algorithm inputs, cost units, and counterexamples. The associated audit report states which checks were actually executed and does not label a cache-assisted extracted build as a cold dependency rebuild.

To reproduce, install the pinned toolchain and LaTeX prerequisites, use the packaged instructions, and run from `Nullivance/`:

```
lake build
lake env lean ResearchValidation.lean
lake env lean RecognitionValidation.lean
lake env lean PublicationValidation.lean
lake env lean CompanionValidation.lean
```

The last two files are generated from the theorem anchors in all three reports. The package also contains the manuscript source, bibliography, and generated catalog, so rebuilding the report does not require private chat records or local drive paths. Source-only reproduction may need network access to obtain the pinned dependencies. No credentials, agent conversation logs, or worktree administration files belong in this publication artifact.

15 Discussion, limitations, and conclusion

15.1 What the results mean

The logical and recognition layers answer different questions. The first controls consequence from support values. The second controls conclusions about a hidden state given an observation contract. Their connection requires explicit assumptions; no theorem turns arbitrary reports into factive knowledge. In particular, the initial-observation counterexample identifies a failure of an interface, whereas the scalar probe construction specifies how a stronger interface can remove that failure.

The stopping results isolate another distinction. Infinitely improving precision can determine a boundary value without any finite history certifying it. A period bound changes the available specification, not merely the speed of the same uninformed search. The positive and negative theorems are consistent because their input contracts differ. This separation is useful when designing software that must report uncertainty instead of silently treating timeout as a negative answer.

15.2 Outstanding obligations

A central outstanding obligation is to obtain or validate a usable period or tail certificate from an independently justified application source. It cannot in general be manufactured from the same finite observations whose negative result needs justification. Generator-description verification and assumptions about a physical generating process are different tasks.

Other boundaries remain: unrestricted infinite-domain quantified completeness; validity across all assignments beyond the supplied-assignment equality interfaces; arbitrary structural dimensions and changing-state observations; concrete adequate evidence maps; correlated observation errors; combined serialization and application-security contracts; bit complexity and physical resource bounds; and empirical validation of any cognitive interpretation. Some narrower obligations have already been closed by later records. This list therefore describes current scope rather than repeating every historic "next step" as if it were still open.

Potential applications include checking whether an observation interface supports a requested distinction, returning explicit alternatives when it does not, validating cached summaries against source data, and reporting certified stopping only under justified assumptions. These are mathematically motivated opportunities, not measured superiority over existing systems. No benchmark or human/AI cognitive experiment is reported here.

15.3 Conclusion

The main research contribution of this collection is a traceable formal chain: precise semantics; scoped completeness and equality algorithms; observation loss counterexamples; exact finite

recognition under a specified probe model; source-backed, checked implementations; and a matched impossibility/positive cutoff pair for periodic boundary contact. The introductory comparison identifies relevant established ideas but does not establish priority for the full construction. We do not claim that these general ideas originated with this artifact or that the complete problem of recognition has been solved.

Availability and author declarations

The project repository is https://github.com/lamtung0487-droid/nullivance-logic. The attached source archive identifies this snapshot independently of whether that repository has subsequently advanced. The artifact is distributed under the project's CC BY 4.0 license. The author contact is given on the title page. The version family is https://doi.org/10.5281/zenodo.18309848; the record labeled DR0149-r3 identifies this collection specifically. An archival deposit is not external peer review or empirical certification. No affiliation, funding, or competing-interest statement is inferred from the software artifact.

A Collection identity and relation to earlier editions

This edition changes exposition, bibliography, version labeling, and verification tooling, not the mathematical baseline through DR-0149: `48bd5f4260fd7982176d7a2060be4e956e1fd49c`. Lean and mathlib remain pinned to 4.32.1. Exact packaged bytes are identified by the source manifest and successful verification receipt, not by the baseline commit alone.

Edition	How it should be read
Original version 1	Historical motivation and exploratory formulation; not evidence that every philosophical or physical interpretation was proved.
v0.7.1	Archived propositional and finite-domain papers with their original formalization status; preserved at their original record.
DR0149-r2	Prior synthesis and reproducible artifact. Its bundled core PDF included later formalization updates despite retaining the old cover label.
DR0149-r3	This consistently labeled three-part collection. Part II retains those updates and identifies them; Part III retains its mathematics with revised presentation. The overview is rewritten around questions, proofs, and limits.

Current formal definitions take precedence for current claims. In particular, exact zero is not a small positive intensity; independent support coordinates are not complementary probabilities; a proposed latent structure is not an observed physical cause. Earlier broad comparisons with classical logic do not establish limitations of classical mathematics. These clarifications preserve historical provenance without elevating exploratory claims to theorems.

B Complete chronological coverage through DR-0149

The following catalog preserves every design-record identifier and title in the source baseline. It is a coverage index, not 149 claims of novelty or a claim that every historical proposed statement was proved. Records include definitions, counterexamples, repaired proof routes, algorithms, and cost refinements. The accompanying machine-readable catalog lists original record paths, SHA-256 identities, and explicitly mentioned Lean files when present. Historical verification counts in those records are not substituted for the fresh publication checks.

Record	Topic
DR-0001	Propositional language of NPL
DR-0002	Two-channel continuous semantics and connective clauses
DR-0003	Threshold projection, meta-signs, and the consequence relation
DR-0004	Lean encoding choices and the continuous-layer gap
DR-0005	The four-signed tableau calculus is the canonical proof system
DR-0006	Formalizing the generative tier (Tier 1)
DR-0007	Finite-domain quantified NPL first
DR-0008	Finite-domain quantified tableau first
DR-0009	Equality closure repair for finite-domain quantified tableau
DR-0010	Finite grounding bridge for quantified completeness
DR-0011	Full recursive extensional finite-domain tableau
DR-0012	Constrained grounding completeness for finite-domain tableau
DR-0013	Core extensional tableau and replay trace
DR-0014	Fixed-signature layer for finite quantified NPL
DR-0015	Epistemic statuses for definitions and conventions
DR-0016	Exact continuous-semantics encoding bridges
DR-0017	Verified operational reference search
DR-0018	Progress-scheduler independence without fairness
DR-0019	Refutation of the head-sensitive replay bridge
DR-0020	Membership-selecting replay certificate
DR-0021	Arbitrary-domain quantified semantic foundation
DR-0022	Collision-sensitive cost and finite-domain certificates
DR-0023	Open equality: assignment partitions and fresh capacity
DR-0024	Executable free-assignment encoding
DR-0025	Compact open orbit initialization
DR-0026	Exact finite open equality and checked admission
DR-0027	Recognition, observation loss, and extension criteria
DR-0028	Proposed scalar active probes and their boundaries
DR-0029	Bounded-noise recognition and a rational decoder
DR-0030	Sound abstaining probe classifier without a margin promise
DR-0031	Exact feasibility and completeness of the abstaining classifier
DR-0032	Executable rational recognition witnesses
DR-0033	Exact same-state observation histories
DR-0034	Complete affirmation for finite observation histories
DR-0035	Complete history classification and executable witnesses
DR-0036	Infinite histories need not yield finite determination
DR-0037	Compactness and sharp two-observation inconsistency certificates
DR-0038	Executable small conflict certificates
DR-0039	Sourced linear history summaries and indexed conflict certificates
DR-0040	Finite recognition: attained boundaries versus limiting information
DR-0041	Persistence of finite recognition certificates
DR-0042	Executable bounded recognition-certificate search
DR-0043	Earliest certificate and exact abstract test count
DR-0044	Incremental, resumable recognition-certificate search
DR-0045	Exact finite refutation and two-record certificates
DR-0046	Endpoint classifier and composed comparison budget
DR-0047	Executable indexed refutation certificates
DR-0048	Refutation export from saved search state
DR-0049	Search plus certificate-export comparison budget
DR-0050	Authenticated index-only source deduplication
DR-0051	Finite stopping from a positive-intensity margin
DR-0052	Computable reciprocal-error stopping budget
DR-0053	Affirmative stopping with attained clamps and theta separation
DR-0054	Authenticated finite affirmative certificates
DR-0055	Affirmative export from a saved search cursor
DR-0056	Counted affirmative export and search composition

Record	Topic
DR-0057	Reusing affirmative guard comparisons
DR-0058	Authenticated index-only affirmative export
DR-0059	Computable authentication of supplied source boxes
DR-0060	Checked continuation from an imported cursor
DR-0061	Affirmative certificates after checked continuation
DR-0062	Refutation certificates after checked continuation
DR-0063	Exact-one certificates and joint operational outcomes
DR-0064	Validation-inclusive continuation counters
DR-0065	Counted index-only refutation and joint pipeline
DR-0066	Sharing final-export consistency checks
DR-0067	Field-level charged-node budget for cursor validation
DR-0068	Thread field-level cursor charges through the joint pipeline
DR-0069	Explicit short-circuit equality for source records
DR-0070	Source short-circuit comparison through checked joint certificates
DR-0071	No source short-circuit saving on an accepted cursor
DR-0072	Finite digests alone cannot authenticate every saved cursor
DR-0073	Finite-digest failure persists for feasible alpha/Theta histories
DR-0074	Proof-carrying in-process cursor continuation
DR-0075	Checked entry from untrusted saved cursors
DR-0076	Terminal-safe trusted sessions and cumulative schedule bound
DR-0077	Checked import followed by a trusted finite schedule
DR-0078	Active-only checkpoint and revalidation boundary
DR-0079	Canonical typed-byte checkpoint codec
DR-0080	Versioned ByteArray checkpoint envelope and size gate
DR-0081	A first-stage numerical bound for checkpoint decoding
DR-0082	A non-vacuous bridge from finite histories to factive knowledge
DR-0083	Conditional evidence-to-knowledge bridge and two obstructions
DR-0084	Init-only FOUR reports cannot exactly detect quasivance
DR-0085	Each of the two existing active probes is individually insufficient
DR-0086	Finitely many positive FOUR thresholds remain insufficient
DR-0087	An infinite shrinking-threshold trace identifies zero intensity
DR-0088	Whole countable threshold signature recognizes scalar quasivance
DR-0089	Three threshold levels suffice under an explicit scalar margin
DR-0090	Finite one-sided intensity refutation without a uniform margin
DR-0091	Finite witnesses of scalar structure nonneutrality
DR-0092	No finite prefix affirms scalar quasivance
DR-0093	A neutral nonquasivant boundary cannot be refuted by finite prefixes
DR-0094	Every zero-intensity scalar prefix admits a quasivant state
DR-0095	Exact eventual finite-prefix refutation criterion
DR-0096	The zero-length full prefix already has a refutation bit
DR-0097	Exact refutation criterion for every full finite prefix
DR-0098	Compressing full-prefix intensity refutation
DR-0099	Single-threshold summary of finite-prefix refutation
DR-0100	Supplied-list refutation and exact traversal cost
DR-0101	Incremental threshold endpoint and update cost
DR-0102	Imported threshold-cache validation and replay cost
DR-0103	Proof-carrying threshold import and preserved update invariant
DR-0104	Checked import followed by a finite online trace
DR-0105	First-hit threshold stopping and scalar persistence
DR-0106	Checked stopping, chunk continuation and explicit offset cost
DR-0107	Retained consumption and repeated-chunk cost
DR-0108	Checked import followed by certified chunk stopping
DR-0109	Validation of stopping-checkpoint metadata
DR-0110	Continuing an accepted stopping certificate
DR-0111	Finite continuation sessions and a separate cost ledger
DR-0112	Ordered delivery bridges finite sessions to eventual refutation

Record	Topic
DR-0113	Explicit rational-margin observation and delivery budgets
DR-0114	Feasibility-guarded threshold certificates from error histories
DR-0115	Searching partial intensity certificates without false completion
DR-0116	Lazy certificates and safe replay after observation refinement
DR-0117	Shared materialized summaries for partial threshold replay
DR-0118	Length-carrying summaries and finite-session cost
DR-0119	Reusing resolved thresholds under feasible refinement
DR-0120	Counted selective-cache initialization and finite sessions
DR-0121	Checked import of the combined selective cache
DR-0122	Finite unknown-exit potential and qualitative stabilization
DR-0123	Precision-delivery deadlines for separated intensity thresholds
DR-0124	Adaptive prefix resolution without a prior margin
DR-0125	Incremental all-threshold resolution search
DR-0126	Exact cache and cost composition across search budgets
DR-0127	Checking an imported resolution run against finite source data
DR-0128	Replaying only the observations a completed search needs
DR-0129	Charged work of a replay that runs out of observations
DR-0130	Resume replay after an observation shortage
DR-0131	What must be checked before resuming an imported pending phase?
DR-0132	Receive the starting prefix incrementally
DR-0133	A sufficient sourced summary for incomplete initialization
DR-0134	When does a finite history decide an exact intensity boundary?
DR-0135	Exact eventual stopping for a finite list of intensity thresholds
DR-0136	No finite observational certificate of permanent non-resolution in general
DR-0137	A sharp cutoff given a finite generator of boundary contacts
DR-0138	Finite validation of a candidate contact generator
DR-0139	Checkable finite relation certificates for generator traces
DR-0140	Proof-carrying symbolic relations for infinite counter generators
DR-0141	An executable checker for affine relations of periodic counters
DR-0142	Exact trace decision through a finite residue quotient
DR-0143	Stride-sensitive synchronized cycle cutoff
DR-0144	Verified arithmetic setup and scanner work
DR-0145	Logarithmic Euclidean remainder-count bounds
DR-0146	Skip cycle preparation on an immediate disagreement
DR-0147	Periodic contact decision and the unknown-period prefix limit
DR-0148	Adaptive querying does not remove the unknown-period limit
DR-0149	Observation-only decision under a certified period bound

References

[1] Nuel D. Belnap, Jr. A useful four-valued logic. In J. Michael Dunn and George Epstein, editors, *Modern Uses of Multiple-Valued Logic*, pages 5–37. D. Reidel, Dordrecht, 1977. https://doi.org/10.1007/978-94-010-1161-7_2.

[2] Randal E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986. https://doi.org/10.1109/TC.1986.1676819.

[3] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, pages 238–252. ACM, 1977. https://doi.org/10.1145/512950.512973.

[4] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.

[5] J. Michael Dunn. Intuitive semantics for first-degree entailments and "coupled trees". *Philosophical Studies*, 29(3):149–168, 1976. https://doi.org/10.1007/BF00373152.

[6] Melvin Fitting. Bilattices and the semantics of logic programming. *Journal of Logic Programming*, 11(2):91–116, 1991. https://doi.org/10.1016/0743-1066(91)90014-G.

[7] Joseph Y. Halpern and Yoram Moses. Knowledge and common knowledge in a distributed environment. *Journal of the ACM*, 37(3):549–587, 1990. Author-deposited version: https://arxiv.org/abs/cs/0006009.

[8] Luc Jaulin and Eric Walter. Set inversion via interval analysis for nonlinear bounded-error estimation. *Automatica*, 29(4):1053–1064, 1993. https://doi.org/10.1016/0005-1098(93)90106-4.